In [1]:
```
import pandas as pd
import numpy as np
import tensorflow as tf
```

In [2]:
```
df = pd.read_csv(r'C:\Users\sande\OneDrive\Desktop\Naresh IT\data science and ai\Ar
```

In [3]:
```
df.head(4)
```

Out[3]:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	B
0	1	15634602	Hargrave	619	France	Female	42	2	
1	2	15647311	Hill	608	Spain	Female	41	1	83
2	3	15619304	Onio	502	France	Female	42	8	159
3	4	15701354	Boni	699	France	Female	39	1	

In [4]:
```
X = df.iloc[:,3:-1].values
y = df.iloc[:,-1].values
```

In [5]:
```
X
```

Out[5]:
```
array([[619, 'France', 'Female', ..., 1, 1, 101348.88],
       [608, 'Spain', 'Female', ..., 0, 1, 112542.58],
       [502, 'France', 'Female', ..., 1, 0, 113931.57],
       ...,
       [709, 'France', 'Female', ..., 0, 1, 42085.58],
       [772, 'Germany', 'Male', ..., 1, 0, 92888.52],
       [792, 'France', 'Female', ..., 1, 0, 38190.78]], dtype=object)
```

In [6]:
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X[:,2] = le.fit_transform(X[:,2])
```

In [7]:
```
X
```

Out[7]:
```
array([[619, 'France', 0, ..., 1, 1, 101348.88],
       [608, 'Spain', 0, ..., 0, 1, 112542.58],
       [502, 'France', 0, ..., 1, 0, 113931.57],
       ...,
       [709, 'France', 0, ..., 0, 1, 42085.58],
       [772, 'Germany', 1, ..., 1, 0, 92888.52],
       [792, 'France', 0, ..., 1, 0, 38190.78]], dtype=object)
```

In [8]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

ct = ColumnTransformer(transformers=[('encoder',OneHotEncoder(),[1])],remainder='pa
X = np.array(ct.fit_transform(X))
```

In [9]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X = sc.fit_transform(X)
```

In [10]:
```
X
```

Out[10]:
```
array([[ 0.99720391, -0.57873591, -0.57380915, ...,  0.64609167,
         0.97024255,  0.02188649],
       [-1.00280393, -0.57873591,  1.74273971, ..., -1.54776799,
         0.97024255,  0.21653375],
       [ 0.99720391, -0.57873591, -0.57380915, ...,  0.64609167,
        -1.03067011,  0.2406869 ],
       ...,
       [ 0.99720391, -0.57873591, -0.57380915, ..., -1.54776799,
         0.97024255, -1.00864308],
       [-1.00280393,  1.72790383, -0.57380915, ...,  0.64609167,
        -1.03067011, -0.12523071],
       [ 0.99720391, -0.57873591, -0.57380915, ...,  0.64609167,
        -1.03067011, -1.07636976]])
```

In [11]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.20, random_state
```

In [12]:
```
ann = tf.keras.models.Sequential()

ann.add(tf.keras.layers.Dense(units = 6,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 6,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 5,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 4,activation = 'relu'))
ann.add(tf.keras.layers.Dense(units = 1,activation = 'sigmoid'))
```

In [13]:
```
ann.compile(optimizer = 'adamax',loss = 'binary_crossentropy',metrics = ['accuracy'
```

In [14]:
```
ann.fit(X_train,y_train,batch_size = 32,epochs = 20,validation_data=(X_test,y_test)
```

```
Epoch 1/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.6301 - loss: 0.6910 - val_acc
uracy: 0.7970 - val_loss: 0.6299
Epoch 2/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.5839 - val_acc
uracy: 0.7975 - val_loss: 0.5427
Epoch 3/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.5118 - val_acc
uracy: 0.7975 - val_loss: 0.5040
Epoch 4/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4895 - val_acc
uracy: 0.7975 - val_loss: 0.4930
Epoch 5/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4795 - val_acc
uracy: 0.7975 - val_loss: 0.4828
Epoch 6/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4699 - val_acc
uracy: 0.7975 - val_loss: 0.4720
Epoch 7/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4604 - val_acc
uracy: 0.7975 - val_loss: 0.4618
Epoch 8/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4520 - val_acc
uracy: 0.7975 - val_loss: 0.4527
Epoch 9/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4442 - val_acc
uracy: 0.7975 - val_loss: 0.4451
Epoch 10/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4381 - val_acc
uracy: 0.7975 - val_loss: 0.4388
Epoch 11/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4327 - val_acc
uracy: 0.7975 - val_loss: 0.4345
Epoch 12/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4291 - val_acc
uracy: 0.7975 - val_loss: 0.4309
Epoch 13/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4259 - val_acc
uracy: 0.7975 - val_loss: 0.4277
Epoch 14/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7960 - loss: 0.4231 - val_acc
uracy: 0.7975 - val_loss: 0.4245
Epoch 15/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8060 - loss: 0.4204 - val_acc
uracy: 0.8360 - val_loss: 0.4220
Epoch 16/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8251 - loss: 0.4180 - val_acc
uracy: 0.8365 - val_loss: 0.4191
Epoch 17/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8259 - loss: 0.4155 - val_acc
uracy: 0.8370 - val_loss: 0.4180
Epoch 18/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8270 - loss: 0.4135 - val_acc
uracy: 0.8385 - val_loss: 0.4145
Epoch 19/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8278 - loss: 0.4113 - val_acc
```

```
uracy: 0.8395 - val_loss: 0.4132
Epoch 20/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8275 - loss: 0.4091 - val_acc
uracy: 0.8390 - val_loss: 0.4116
```

Out[14]:
```
<keras.src.callbacks.history.History at 0x16b22a86810>
```

In [15]:
```
y_pred = ann.predict(X_test)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

In [16]:
```
y_pred = (y_pred > 0.5)
```

In [17]:
```
print(np.concatenate((y_pred.reshape(len(y_pred),1),y_test.reshape(len(y_test),1)),
```

```
[[0 0]
 [0 1]
 [0 0]
 ...
 [0 0]
 [0 0]
 [0 0]]
```

In [18]:
```
from sklearn.metrics import accuracy_score,confusion_matrix

ac = accuracy_score(y_test,y_pred)
ac
```

Out[18]:
```
0.839
```

In [19]:
```
cm = confusion_matrix(y_test,y_pred)
cm
```

Out[19]:
```
array([[1529,   66],
       [ 256,  149]], dtype=int64)
```